

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Криптографические методы обеспечения информационной безопасности»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

«Основные структурные элементы блочного симметричного алгоритма DES»

Выполнили:

Бардышев Артём Антонович,
студент группы N3346



(подпись)

Проверил:

Чешев Никита Игоревич,
инженер, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Введение	3
1 Визуализация алгоритмов шифрования	4
1.1 Изучение структуры DES	4
1.2 Изучение лавинного эффекта	5
1.3 Изучение режимов шифрования	6
2 Padding Oracle атака	8
2.1 Изучение Padding Oracle атаки	8
2.2 Ручное выполнение одного раунда DES	9
2.2.1 Перестановка с расширением E	9
2.2.2 Побитовое сложение с ключом (XOR)	10
2.2.3 Операция подстановки (S-box)	10
2.2.4 Прямая перестановка P	13
3 Работа с OpenSSL	14
3.1 Просмотр доступных алгоритмов	14
3.2 Шифрование файла	15
3.3 Анализ параметров шифрования	15
3.4 Дешифрование	15
Заключение	17

ВВЕДЕНИЕ

Цель работы – изучить основные принципы работы алгоритмы DES

Задачи практической работы:

1. Проанализировать эмуляцию алгоритма DES и примитивных атак на шифр, используя Cryptool 2. Выделить основные необходимые настройки шифра и требуемые ограничения на параметры.
2. Выполнение 1 цикла раундовой функции алгоритма DES вручную, то есть выполнение всех функций, входящих в раундовую функцию DES для фиксированного входного двоичного вектора с отображением промежуточных значений шифрования. Также для подробного изучения шифра может быть использована программная реализация 1 раунда (или полной системы) DES в режиме отладки с выводом промежуточных значений шифрования.
3. Анализ принципов использования криптосистемы в современных приложениях на примере библиотеки openssl.

1 ВИЗУАЛИЗАЦИЯ АЛГОРИТМОВ ШИФРОВАНИЯ

1.1 Изучение структуры DES

1. Откройте СrypTool 2

2. Перейдите в Templates - Cryptography - Modern - Symmetric - DES Visualization

3. Введите свои входные данные (открытый текст) и криптографический ключ

Раундовая функция (F-функция)

1. Перестановка с расширением E (Expansion Permutation)

- Расширяет 32-битный блок до 48 бит

- Дублирует некоторые биты для соответствия размеру раундового ключа

2. Сложение с ключом (XOR с раундовым ключом)

- Побитовое сложение по модулю 2 расширенного блока с 48-битным раундовым

КЛЮЧОМ

3. Нелинейная замена (S-box)

- 8 S-боксов, каждый преобразует 6 бит в 4 бита

- Это основная нелинейная операция DES

- Изучите структуру каждого S-бокса

4. Прямая перестановка P (Permutation)

- Переставляет 32 бита после S-боксов

- Обеспечивает диффузию

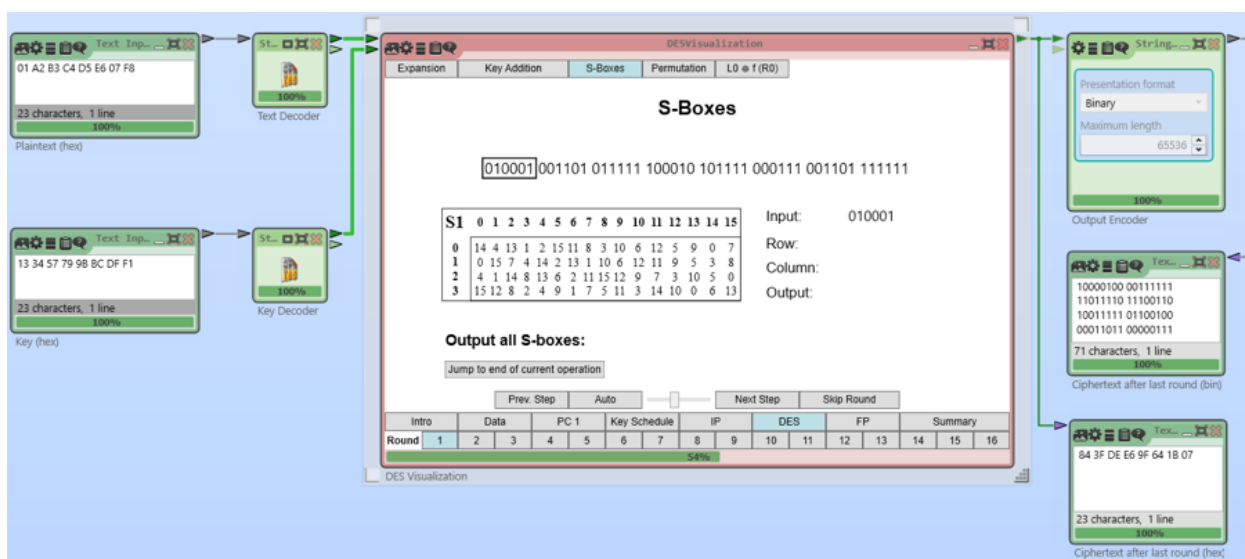


Рисунок 1 – DES

1.2 Изучение лавинного эффекта

1. Перейдите в Templates - Cryptanalysis - Modern - Avalanche (DES)
2. Введите открытый текст и ключ
3. Зашифруйте текст и зафиксируйте результат

Эксперименты:

1. Изменение одного бита открытого текста:
 - Измените один бит в открытом тексте
 - Зашифруйте снова
 - Подсчитайте процент изменившихся битов в шифротексте
 - Ожидаемый результат: должно измениться примерно 50% битов (признак хорошего лавинного эффекта)
2. Изменение одного бита ключа:
 - Измените один бит в ключе
 - Зашифруйте тот же открытый текст
 - Подсчитайте процент изменившихся битов
 - Сравните с предыдущим экспериментом
3. Изменение нескольких битов:
 - Измените 2, 3, 4 бита в разных позициях
 - Проанализируйте характер группировки искаженных значений
 - Отметьте скорость распространения изменений
4. Искажения в промежуточных значениях:
 - Внесите искажения в промежуточные значения на 10, 12, 14 раундах
 - Проследите, каких значений достигнет лавинный эффект
 - Оцените, насколько быстро распространяются изменения

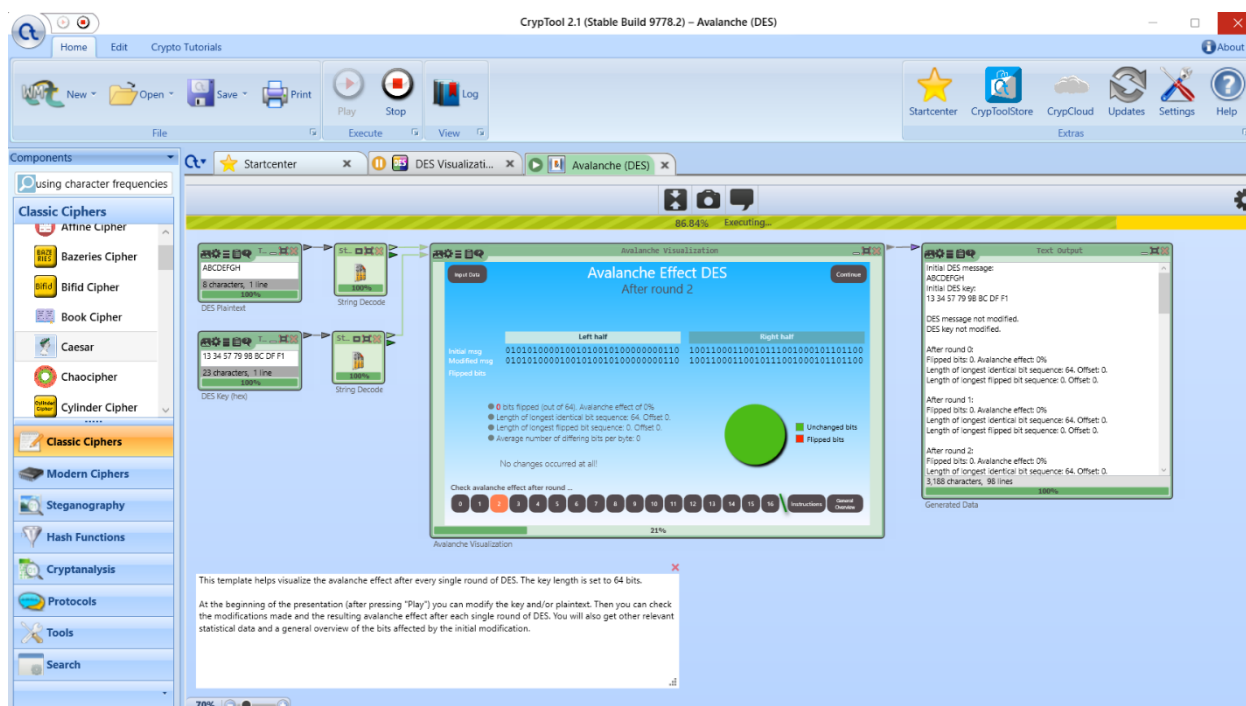


Рисунок 2 – Лавинный эффект

1.3 Изучение режимов шифрования

1. Перейдите в Templates - Cryptography - Modern - Block Modes of Symmetric Ciphers
2. Изучите различные режимы шифрования

Режимы для изучения:

ECB (Electronic Codebook)

- Принцип: каждый блок шифруется независимо
- Опасность: одинаковые блоки открытого текста дают одинаковые блоки шифротекста

шифротекста

- Проблема: уязвим к атакам на основе анализа паттернов
 - Визуализация: зашифруйте изображение в режиме ECB - будут видны паттерны
- CBC (Cipher Block Chaining)

- Принцип: каждый блок XOR-ится с предыдущим шифротекстом перед шифрованием

- Требуется: вектор инициализации (IV)

- Преимущества: скрывает паттерны, обеспечивает диффузию

- Недостатки: последовательная обработка (нельзя распараллелить)

CFB (Cipher Feedback)

- Принцип: шифр работает как потоковый, используя предыдущий шифротекст
- Применение: когда нужна побайтовая обработка

OFB (Output Feedback)

- Принцип: генерирует ключевой поток независимо от открытого текста
- Особенность: ошибки не распространяются

CTR (Counter)

- Принцип: использует счетчик, который шифруется для получения ключевого потока

- Преимущества: можно распараллелить, не требует IV (но рекомендуется)
- Применение: широко используется в современных системах

GCM (Galois/Counter Mode)

- Принцип: комбинация CTR с аутентификацией
- Преимущества: обеспечивает и шифрование, и аутентификацию
- Применение: современный стандарт для сетевых протоколов

2 **PADDING ORACLE ATAKA**

2.1 **Изучение Padding Oracle атаки**

1. Используйте шаблон для демонстрации Padding Oracle атаки (если доступен в CrypTool 2)

2. Или изучите принцип атаки теоретически

Принцип атаки:

- Основана на проверке корректности padding (дополнения) при дешифровании
- Атакующий отправляет модифицированный шифротекст на сервер
- Сервер отвечает, корректен ли padding (ошибка или успех)
- По ответам сервера атакующий восстанавливает открытый текст

Этапы атаки:

1. Атакующий имеет шифротекст C (состоит из блоков $C_1, C_2, \dots, C_n$)
2. Модифицирует предпоследний блок C_{n-1}
3. Отправляет модифицированный шифротекст на сервер
4. Анализирует ответ сервера (корректен ли padding)
5. По ответам восстанавливает промежуточное значение
6. Вычисляет открытый текст

Примеры применения в реальной жизни:

- Атака на веб-приложения, использующие CBC режим
- Атака на протоколы, не проверяющие целостность перед дешифрованием
- Известные уязвимости: ASP.NET, Java, Ruby on Rails (в прошлом)

Защита:

- Использование аутентифицированного шифрования (AEAD)
- Проверка целостности перед дешифрованием
- Использование режимов с встроенной аутентификацией (GCM, CCM)

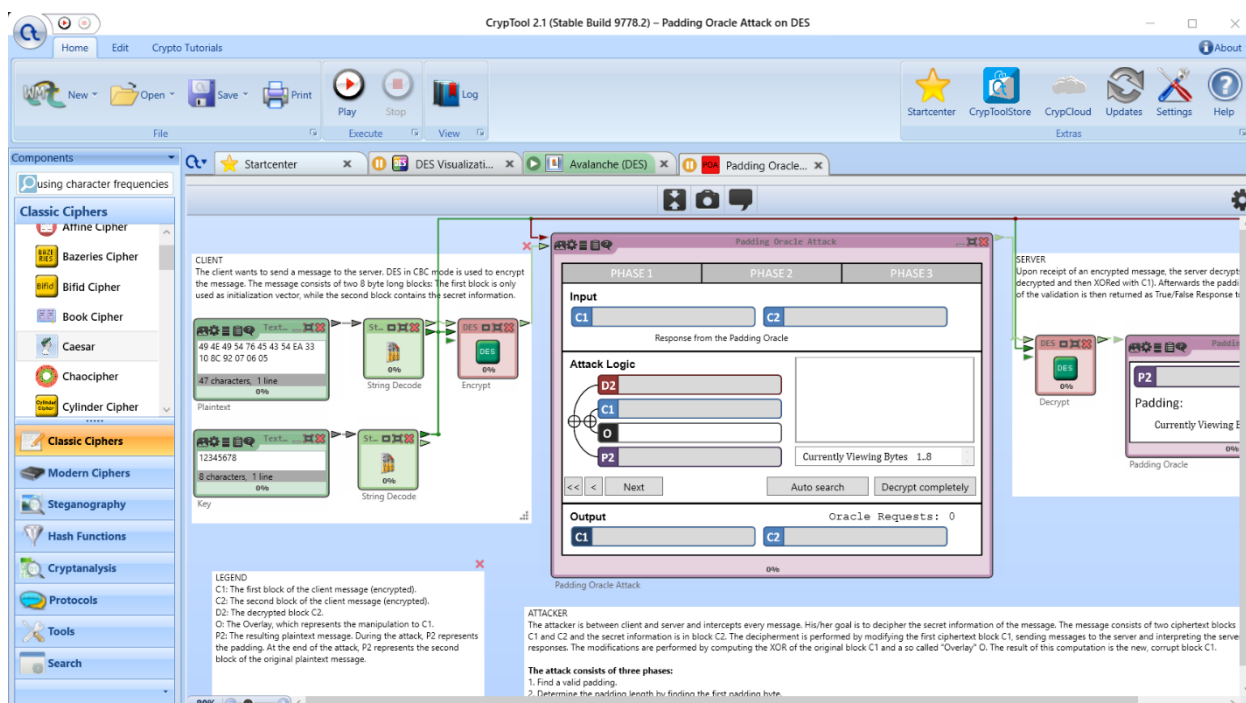


Рисунок 3 – Padding Oracle атака

2.2 Ручное выполнение одного раунда DES

Исходные данные

Входной блок (правая половина блока, 32 бита):

R = 1011 0100 1100 0111 0010 1110 1001 1011

В шестнадцатеричном виде: `B4C7 2E9B`

Раундовый ключ (48 бит):

K = 1101 0110 1010 0011 1100 1001 0111 1010 0011 0100 1110 1011

В шестнадцатеричном виде: `D6A3 C97A 34EB`

2.2.1 Перестановка с расширением E

Таблица расширения E:

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

Выполнение расширения:

Входной блок R (32 бита):

Позиция: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27
28 29 30 31 32

Значение: 1 0 1 1 0 1 0 0 1 1 0 0 0 1 1 1 0 0 1 0 1 1 1 0 1 0 0 1 1 0 1 1

Применяем таблицу E (читаем позиции из таблицы и берем соответствующие биты):

Результат расширения E(R) (48 бит):

Позиция E: 32 1 2 3 4 5 4 5 6 7 8 9 8 9 10 11 12 13 12 13 14 15 16 17 16 17 18
19 20 21 20 21 22 23 24 25 24 25 26 27 28 29 28 29 30 31 32 1

Бит: 1 1 0 1 1 0 0 1 1 0 0 0 0 0 1 1 0 1 1 1 0 0 1 0 0 1 0 1 1 1 1 1 0
1 0 0 0 0 1 0 0 1 1 0 1 1 1 1

E(R) в двоичном виде:

1101 1001 1000 0011 0111 0010 0111 1110 0001 0011 0111 1111

E(R) в шестнадцатеричном виде: `D9 83 72 7E 13 7F`

2.2.2 Побитовое сложение с ключом (XOR)

Входные данные:

- E(R) = `1101 1001 1000 0011 0111 0010 0111 1110 0001 0011 0111 1111`

- K = `1101 0110 1010 0011 1100 1001 0111 1010 0011 0100 1110 1011`

Выполнение XOR (побитово):

E(R): 1101 1001 1000 0011 0111 0010 0111 1110 0001 0011 0111 1111

K: 1101 0110 1010 0011 1100 1001 0111 1010 0011 0100 1110 1011

XOR: 0000 1111 0010 0000 1011 1011 0000 0100 0010 0111 0001 0100

Результат E(R) $\oplus$ K (48 бит):

0000 1111 0010 0000 1011 1011 0000 0100 0010 0111 0001 0100

В шестнадцатеричном виде: `0F 20 BB 04 27 14`

2.2.3 Операция подстановки (S-box)

Разделение на 8 групп по 6 бит:

Результат XOR: `0000 1111 0010 0000 1011 1011 0000 0100 0010 0111 0001 0100`

Разделяем на группы:

1. Группа 1: `000011` (биты 1-6)

2. Группа 2: `110010` (биты 7-12)

3. Группа 3: `000010` (биты 13-18)
4. Группа 4: `111011` (биты 19-24)
5. Группа 5: `000001` (биты 25-30)
6. Группа 6: `000010` (биты 31-36)
7. Группа 7: `011100` (биты 37-42)
8. Группа 8: `010100` (биты 43-48)

Обработка каждой группы через S-боксы:

Примечание: В предоставленном изображении показан один S-бкс. Для полного DES нужно использовать все 8 S-боксов ($S_1, S_2, \dots, S_8$). В данном примере будем использовать предоставленный S-бкс для демонстрации принципа.

Предоставленный S-бкс:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Row 0:	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
Row 1:	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
Row 2:	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
Row 3:	15	12	8	2	4	9	1	7	5	11	3	14	10	0	5	13

Правило:

- Первый и последний бит (биты 1 и 6) определяют строку (0-3)
- Средние 4 бита (биты 2-5) определяют столбец (0-15)

Обработка групп:

Группа 1: `000011`

- Бит 1 (первый) = 0, Бит 6 (последний) = 1 - Строка = `01` (двоичное) = 1 (десятичное)
- Биты 2-5 = `0001` - Столбец = 1 (десятичное)
- $S_1[1][1] = 1$ (из S-бокса, строка 1, столбец 1)
- Результат: `0001` (4 бита)

Группа 2: `110010`

- Бит 1 = 1, Бит 6 = 0 - Строка = `10` = 2
- Биты 2-5 = `1001` - Столбец = 9
- $S_2[2][9] = 12$ (из S-бокса, строка 2, столбец 9)
- Результат: `1100` (4 бита)

Группа 3: `000010`

- Бит 1 = 0, Бит 6 = 0 - Строка = `00` = 0
- Биты 2-5 = `0001` - Столбец = 1
- $S_3[0][1] = 4$ (из S-бокса, строка 0, столбец 1)

- Результат: `0100` (4 бита)

Группа 4: `111011`

- Бит 1 = 1, Бит 6 = 1 - Строка = `11` = 3

- Биты 2-5 = `1101` - Столбец = 13

- $S_4[3][13] = 0$ (из S-блока, строка 3, столбец 13)

- Результат: `0000` (4 бита)

Группа 5: `000001`

- Бит 1 = 0, Бит 6 = 1 - Строка = `01` = 1

- Биты 2-5 = `0000` - Столбец = 0

- $S_5[1][0] = 0$ (из S-блока, строка 1, столбец 0)

- Результат: `0000` (4 бита)

Группа 6: `000010`

- Бит 1 = 0, Бит 6 = 0 - Строка = `00` = 0

- Биты 2-5 = `0001` - Столбец = 1

- $S_6[0][1] = 4$ (из S-блока, строка 0, столбец 1)

- Результат: `0100` (4 бита)

Группа 7: `011100`

- Бит 1 = 0, Бит 6 = 0 - Строка = `00` = 0

- Биты 2-5 = `1110` - Столбец = 14

- $S_7[0][14] = 0$ (из S-блока, строка 0, столбец 14)

- Результат: `0000` (4 бита)

Группа 8: `010100`

- Бит 1 = 0, Бит 6 = 0 - Строка = `00` = 0

- Биты 2-5 = `1010` - Столбец = 10

- $S_8[0][10] = 6$ (из S-блока, строка 0, столбец 10)

- Результат: `0110` (4 бита)

Объединение результатов S-блоков (32 бита):

Группа 1: 0001

Группа 2: 1100

Группа 3: 0100

Группа 4: 0000

Группа 5: 0000

Группа 6: 0100

Группа 7: 0000

Группа 8: 0110

Результат: 0001 1100 0100 0000 0000 0100 0000 0110

Результат S-блоков (32 бита):

0001 1100 0100 0000 0000 0100 0000 0110

В шестнадцатеричном виде: `1C 40 04 06`

2.2.4 Прямая перестановка P

Таблица перестановки P:

16 7 20 21 29 12 28 17

1 15 23 26 5 18 31 10

2 8 24 14 32 27 3 9

19 13 30 6 22 11 4 25

Выполнение перестановки:

Входной блок после S-блоков (32 бита):

Позиция: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27
28 29 30 31 32

Значение: 0 0 0 1 1 1 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 1 0 0 0 0

Применяем таблицу P (читаем позиции из таблицы и берем соответствующие биты):

Результат перестановки P (32 бита):

Позиция P: 16 7 20 21 29 12 28 17 1 15 23 26 5 18 31 10 2 8 24 14 32 27 3 9 19 13
30 6 22 11 4 25

Бит: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0

P(S-результат) в двоичном виде:

0000 0000 0000 0010 0000 0000 0001 0000

В шестнадцатеричном виде: `00 00 02 00 00 01 00`

Итоговый результат раундовой функции F(R, K)

Финальный результат (32 бита):

0000 0000 0000 0010 0000 0000 0001 0000

В шестнадцатеричном виде: `00020010`

3 РАБОТА С OPENSSL

3.1 Просмотр доступных алгоритмов

openssl enc -list

```
C:\Users\ububkm>openssl enc -list
Supported ciphers:
-aes-128-cbc          -aes-128-cfb          -aes-128-cfb1
-aes-128-cfb8         -aes-128-ctr          -aes-128-ecb
-aes-128-ofb          -aes-192-cbc          -aes-192-cfb
-aes-192-cfb1         -aes-192-cfb8         -aes-192-ctr
-aes-192-ecb          -aes-192-ofb          -aes-256-cbc
-aes-256-cfb          -aes-256-cfb1         -aes-256-cfb8
-aes-256-ctr          -aes-256-ecb          -aes-256-ofb
-aes128               -aes128-wrap          -aes128-wrap-pad
-aes192               -aes192-wrap          -aes192-wrap-pad
-aes256               -aes256-wrap          -aes256-wrap-pad
-aria-128-cbc         -aria-128-cfb         -aria-128-cfb1
-aria-128-cfb8        -aria-128-ctr         -aria-128-ecb
-aria-128-ofb         -aria-192-cbc         -aria-192-cfb
-aria-192-cfb1        -aria-192-cfb8        -aria-192-ctr
-aria-192-ecb         -aria-192-ofb         -aria-256-cbc
-aria-256-cfb         -aria-256-cfb1        -aria-256-cfb8
-aria-256-ctr         -aria-256-ecb         -aria-256-ofb
-aria128              -aria192              -aria256
-bf                   -bf-cbc               -bf-cfb
-bf-ecb              -bf-ofb               -blowfish
-camellia-128-cbc     -camellia-128-cfb     -camellia-128-cfb1
-camellia-128-cfb8    -camellia-128-ctr     -camellia-128-ecb
-camellia-128-ofb     -camellia-192-cbc     -camellia-192-cfb
-camellia-192-cfb1    -camellia-192-cfb8    -camellia-192-ctr
-camellia-192-ecb     -camellia-192-ofb     -camellia-256-cbc
-camellia-256-cfb     -camellia-256-cfb1    -camellia-256-cfb8
-camellia-256-ctr     -camellia-256-ecb     -camellia-256-ofb
-camellia128          -camellia192          -camellia256
-cast                 -cast-cbc             -cast5-cbc
-cast5-cfb           -cast5-ecb            -cast5-ofb
-chacha20             -des                  -des-cbc
-des-cfb              -des-cfb1             -des-cfb8
-des-ecb              -des-ede              -des-ede-cbc
-des-ede-cfb          -des-ede-ecb          -des-ede-ofb
-des-ede3             -des-ede3-cbc          -des-ede3-cfb
-des-ede3-cfb1        -des-ede3-cfb8        -des-ede3-ecb
-des-ede3-ofb         -des-ofb              -des3
-des3-wrap            -desx                 -desx-cbc
-id-aes128-wrap       -id-aes128-wrap-pad   -id-aes192-wrap
-id-aes192-wrap-pad   -id-aes256-wrap       -id-aes256-wrap-pad
-id-smime-alg-CMS3DESwrap -idea                -idea-cbc
-idea-cfb             -idea-ecb             -idea-ofb
-rc2                  -rc2-128              -rc2-40
-rc2-40-cbc           -rc2-64               -rc2-64-cbc
-rc2-cbc              -rc2-cfb              -rc2-ecb
-rc2-ofb              -rc4                  -rc4-40
-seed                 -seed-cbc             -seed-cfb
-seed-ecb             -seed-ofb             -sm4
-sm4-cbc              -sm4-cfb              -sm4-ctr
-sm4-ecb              -sm4-ofb
```

Рисунок 4 – Доступные алгоритмы шифрования

3.2 Шифрование файла

Выберите минимум 3 подхода шифрования с DES:

- DES в режиме ECB
- DES в режиме CBC
- 3DES (triple DES) в разных вариантах

Примеры команд:

DES-CBC:

```
openssl enc -des-cbc -salt -in example.txt -out example.txt.enc -k "Bardyshev"
```

3DES-CBC:

```
openssl enc -des3 -salt -in example.txt -out example.txt.enc -k "Bardyshev"
```

DES-ECB:

```
openssl enc -des-ecb -salt -in example.txt -out example.txt.enc -k "Bardyshev"
```

Создайте входной файл:

```
echo "Hello world" > example.txt
```

3.3 Анализ параметров шифрования

Флаги и параметры для изучения:

- -salt: добавляет случайную соль к паролю
 - Влияние: каждый раз генерируется разный шифротекст даже для одного пароля
 - Криптостойкость: защита от атак по словарю и rainbow tables
- -a (base64): кодирует вывод в base64
 - Влияние: позволяет передавать бинарные данные через текстовые каналы
 - Криптостойкость: не влияет на стойкость, только на представление
- -k: задает пароль напрямую в командной строке
 - Влияние: удобно, но небезопасно (видно в истории команд)
 - Криптостойкость: пароль должен быть достаточно сложным
- -iter: задает количество итераций для PBKDF2
 - Влияние: увеличивает время вычисления ключа из пароля
 - Криптостойкость: защита от brute-force атак на пароль

3.4 Дешифрование

```
openssl enc -des-cbc -d -salt -in example.txt.enc -out example_decrypted.txt -k "Bardyshev"
```

Проверка:

```
diff example.txt example_decrypted.txt
```

Файлы должны совпадать.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены и проанализированы следующие аспекты алгоритма DES:

1. Структура алгоритма DES:

- Сеть Фейстеля и ее принципы работы
- Раундовая функция F и ее компоненты
- Процедура расширения ключа

2. Криптографические свойства:

- Лавинный эффект и его характеристики
- Свойства конфузии и диффузии
- Влияние изменений входных данных на выход

3. Режимы шифрования:

- ECB, CBC, CFB, OFB, CTR, GCM
- Сравнительный анализ режимов
- Применение в различных сценариях

4. Криптоанализ:

- Padding Oracle атака
- Анализ уязвимостей режимов шифрования

5. Практическое применение:

- Использование OpenSSL для шифрования
- Параметры и опции шифрования
- Влияние параметров на криптостойкость

Выводы:

- Сеть Фейстеля - элегантное решение для симметричного шифрования
- Комбинация линейных и нелинейных операций обеспечивает стойкость
- Каждый компонент раундовой функции выполняет важную роль